Depression Classification Report

Aryan Sharma

October 14, 2024

1 Dataset Description

The dataset used for this classification task is titled *Depression_train.csv*, which contains text data and corresponding labels. The text data includes posts or sentences that express various sentiments or mental states, and the labels indicate whether the post suggests depression or not. The dataset has the following columns:

- **text:** This column contains the textual data, which is the primary feature used for classification. Each entry represents a sentence or a paragraph expressing the user's thoughts or feelings.
- **label:** This column contains binary labels:
  - 0: The post does not indicate depression.
  - 1: The post indicates depression.

The dataset has a total of 5,596 instances with a fairly balanced distribution of the two classes (depression and non-depression).

2 Steps Implemented in Code for Classification

The steps implemented in the code for classification are as follows:

2.1 Loading the Dataset

The dataset is first loaded using the `pandas` library:

```
data = pd.read_csv('/mnt/data/Depression_train.csv')
```

This step loads the CSV file into a DataFrame object, allowing us to manipulate and analyze the dataset. The first few rows of the dataset are displayed to check its structure.

2.2 Data Preprocessing

We split the data into input features (text) and target labels (depression indication). Then, we divide the data into training and testing sets using an 80-20 split. The `train_test_split` function from `sklearn.model_selection` is used for this purpose.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

2.3 Text Vectorization

The text data is converted into a numerical format using the TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer. This transformation converts the text into a matrix of token counts while considering the importance of words based on their frequency across the entire dataset.

```
tfidf = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

2.4 Model Training

We employ the Logistic Regression algorithm for classification. Logistic Regression is well-suited for binary classification tasks and provides reliable performance.

```
model = LogisticRegression()
model.fit(X_train_tfidf, y_train)
```

2.5 Model Evaluation

After training the model, predictions are made on the test data. The accuracy of the model is calculated, and a classification report is generated, showing precision, recall, and F1-score for both classes (depression and non-depression).

```
y_pred = model.predict(X_test_tfidf)
accuracy_score(y_test, y_pred)
classification_report(y_test, y_pred)
```

3 Output Explanation

The model achieved the following results when tested on 5,596 samples:

```
Accuracy: 0.9226
```

This indicates that the model correctly classified 92.26% of the posts as either depression-related or non-depression-related.

The classification report, generated using `sklearn.metrics.classification_report`, provides detailed metrics for both classes (0: non-depression, 1: depression):

```
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.93      0.92      2802
           1       0.93      0.91      0.92      2794

    accuracy                           0.92      5596
   macro avg       0.92      0.92      0.92      5596
weighted avg       0.92      0.92      0.92      5596
```

3.1 Explanation of Metrics

- **Precision:** Precision measures the proportion of true positives (correctly identified depression posts) out of all positive predictions (total posts classified as depression). For class 1 (depression), the precision is 0.93, meaning 93% of posts classified as depression are indeed depression-related.
- **Recall:** Recall indicates the proportion of true positives out of all actual positives. For class 1, the recall is 0.91, meaning the model successfully identified 91% of all depression-related posts.
- **F1-Score:** The F1-score is the harmonic mean of precision and recall. For both classes, the F1-score is 0.92, representing a balanced trade-off between precision and recall.

The overall accuracy of the model is 92.26%, indicating that it performed well in distinguishing between depression and non-depression posts.

4 Conclusion

This report details the steps and methodology used to classify text data related to mental health, specifically depression detection, using Logistic Regression. The model achieved a high accuracy of 92.26%, and the precision, recall, and F1-score for both classes indicate that the classifier is reliable and performs well in identifying depression-related posts. Further improvements could include experimenting with more advanced models like SVM or deep learning approaches such as BERT for text classification.

5 Output Screenshot

```
(nlp_venv) C:\Users\asus\Desktop\NLP>python depression_classification.py
                                                text  label
0  dear american teens question dutch person hear...      0
1  nothing look forward lifei dont many reasons k...      1
2  music recommendations im looking expand playli...      0
3  im done trying feel betterthe reason im still ...      1
4  worried  year old girl subject domestic physic...      1
Accuracy: 0.9226233023588277
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.93      0.92      2802
           1       0.93      0.91      0.92      2794

    accuracy                           0.92      5596
   macro avg       0.92      0.92      0.92      5596
weighted avg       0.92      0.92      0.92      5596
```